

GNN-based Unified Deep Learning



Furkan Pala 

and



Islem Rekik *

BASIRA Lab, Imperial-X (I-X) and Department of Computing, Imperial College
London, London, United Kingdom

Abstract. Deep learning models often struggle to maintain robust generalizability in medical imaging, particularly under *domain-fracture* scenarios where distributional shifts arise due to varying imaging techniques, acquisition protocols, patient populations, demographics, and equipment. In practice, each hospital may need to develop and train distinct models—differing in functionality (i.e., learning task) and morphology including width and depth—to handle their local data distributions. For example, while one hospital may utilize Euclidean architectures such as MLPs and CNNs to process structured tabular data or regular grid-like image data, another hospital may need to deploy non-Euclidean architectures such as graph neural networks (GNNs) to process inherently irregular data like brain connectomes or other graph-structured biomedical information. However, how to train such heterogeneous models coherently across different datasets, in a manner that enhances the generalizability of each model, remains an open and challenging problem. In this paper, we address this issue by introducing a new learning paradigm, namely **unified learning**. To address the topological differences between these heterogeneous architectures, we first encode each model into a graph representation, enabling us to unify these diverse models within a shared graph learning space. Once represented in this space, a GNN guides the optimization of the unified models. By decoupling the parameters of individual deep learning models and controlling them through the unified GNN (uGNN), our approach enables parameter-sharing and knowledge-transfer across *varying* architectures (MLPs, CNNs and GNNs) and distributions, ultimately improving its generalizability. We evaluate our framework on MorphoMNIST and two MedMNIST benchmarks—PneumoniaMNIST and BreastMNIST—and find that our unified learning improves the performance of individual models when trained on unique distributions and tested on mixed ones, thereby demonstrating generalizability to unseen data with strong distributional shifts. Our source code including benchmarks and evaluation datasets is available at <https://github.com/basiralab/uGNN>.¹

* Corresponding author: f.pala23@imperial.ac.uk, i.rekik@imperial.ac.uk, <http://basira-lab.com>, GitHub: <https://github.com/basiralab/uGNN>

¹ This paper has been selected for an **Oral Presentation** at the PRIME MICCAI 2025 workshop. [\[uGNN YouTube Video\]](#).

Keywords: unified learning, heterogeneous deep learning, graph neural networks, domain-fracture, medical imaging

1 Introduction

Deep learning has led to significant advances in medical image analysis, improving accuracy and robustness in diverse tasks such as lesion detection, organ segmentation, and disease classification [1–3]. Despite such successes, achieving robust generalization across varying clinical conditions remains challenging, particularly in *domain-fracture* scenarios [4, 5]. Such scenarios arise when data are drawn from heterogeneous sources—e.g. different scanners, patient populations, or imaging protocols—leading to distributional shifts between the train and test datasets that degrade performance. Conventional strategies, such as domain adaptation or federated learning [6, 7], often assume architectural homogeneity or alignments between models, thus limiting their capacity to handle fundamentally heterogeneous deep learning architectures.

In practice, healthcare institutions develop machine learning models with differing architectures and parametrizations, tailored to their local patient populations or imaging modalities [8]. These heterogeneous architectures can vary from simpler Multi-Layer Perceptrons (MLPs) to more complex Convolutional Neural Networks (CNNs) and Graph Neural Networks (GNNs), each trained under different conditions. Integrating knowledge across these models is challenging, as their internal representations, layer connectivity, and parameters can differ significantly. Existing learning paradigms are not inherently designed to reconcile such differences, leaving a critical gap in translating learned knowledge across diverse models and data distributions.

To address these limitations, we introduce the *unified learning* framework, which unifies the learning process across highly heterogeneous deep learning architectures within a common representational space. Unlike traditional approaches such as transfer learning, knowledge distillation, and ensemble learning, which focus on aligning parameters, distilling knowledge, or combining model outputs, unified learning represents each model—regardless of architecture—as a graph. This graph captures the model’s layers, connections, and parameters as nodes and edges. By embedding these heterogeneous model-graphs into a shared space, a single graph neural network (GNN) drives their learning, optimizing the GNN instead of directly updating individual model parameters.

Hypothesis:

We hypothesize that **unified learning** solves the generalizability of heterogeneous architectures under domain-fracture scenarios, enabling models to adapt robustly to distributional shifts without enforcing architectural uniformity.

The key challenge then becomes how to integrate heterogeneous model representations into a common graph space using a GNN:

Key Challenge:

How can we unify heterogeneous deep learning architectures within a common graph space, and effectively bridge the gap between Euclidean models (MLPs, CNNs) and non-Euclidean models (GNNs), when training across diverse datasets using a GNN?

By formulating heterogeneous models as graphs and training a GNN to orchestrate their parameter updates, we establish a paradigm that overcomes the limitations imposed by architectural homogeneity. This approach harnesses the topological flexibility of graph representations to accommodate diverse model architectures, enabling both efficient knowledge sharing and enhanced adaptation to new domains. In summary, our key contributions are as follows: **(1)** We propose a *unified learning* paradigm that represents heterogeneous deep learning architectures as graphs, enabling a GNN to simulate forward passes and parameter updates for varied models in a common learning space. **(2)** Our unified learning framework allows training Euclidean and non-Euclidean models coherently. **(3)** We address the inherent complexity of domain-fracture scenarios by integrating distinct models and datasets into a single, architecture-agnostic training process, thereby facilitating robust cross-domain generalization. **(4)** We provide empirical evidence on medical imaging benchmarks, demonstrating that unified learning improves performance, stability, and adaptability of heterogeneous models facing distributional shifts.

2 Related Work

Deep learning has revolutionized medical image analysis, enabling breakthroughs in disease detection, segmentation, and classification tasks [1, 2]. However, generalizing across domain-fracture medical imaging datasets, characterized by heterogeneity in imaging protocols, scanner settings, and patient demographics, remains a critical challenge [9]. Traditional learning paradigms such as transfer learning, knowledge distillation, and federated learning have been extensively explored to address domain fracture but are constrained by their inability to accommodate heterogeneous deep learning architectures [10].

Domain Fracture in Medical Imaging. Domain shifts in medical imaging arise due to differences in acquisition settings, modality characteristics, and institutional practices [4]. Federated learning (FL) has emerged as a prominent strategy for collaboratively training models across decentralized datasets while maintaining data privacy [11]. In the context of medical imaging, FL has shown considerable promise for addressing domain shifts and enhancing model generalization across decentralized datasets, making it a rapidly growing area of research with numerous advancements [12, 13]. However, most FL methods assume homogeneous architectures across participating institutions [7]. Similarly, domain adaptation techniques, such as adversarial learning-based approaches, aim to align feature spaces across domains [14, 15]. While effective in some cases, these approaches often require data centralization or strong assumptions about

the underlying data distributions, making them impractical for highly fractured medical imaging data.

Learning Paradigms for Heterogeneous Architectures. The challenge of synergizing and integrating the training of heterogeneous architectures has received limited attention. For instance, knowledge distillation [10] enables the transfer of knowledge from a complex teacher model to a simpler student model but relies on aligning output distributions rather than internal representations. Collaborative learning approaches [16] have explored model aggregation but remain limited to networks with similar architectures. In contrast, our proposed model *unifies* heterogeneous architectures into a common graph-based learning space, enabling shared learning dynamics across disparate networks as well as datasets.

Graph Neural Networks in Medical Imaging. Graph-based methods have gained traction in medical imaging for their ability to model complex spatial and relational structures [17]. Cell-graph representations, which capture cellular interactions and tissue structures as graphs, have shown promise in histopathology and disease characterization tasks [18]. GNNs extend this framework by learning representations over graphs, enabling tasks such as tumor classification [19] and disease progression modeling [20]. However, to the best of our knowledge, GNNs have never been leveraged to harmonize the training and learning of heterogeneous deep neural networks (DNNs).

3 Methodology

This section presents our **uGNN** framework, which unifies the learning of heterogeneous deep learning architectures within a shared graph-based representation space. In our framework, we can train MLPs, CNNs and GNNs coherently, making unified learning a powerful framework to train Euclidean and non-Euclidean models together.

Motivation. Our motivation for using graphs and GNNs stems from the structural heterogeneity of deep learning architectures. Each model type—such as MLPs, CNNs, or GNNs—differs in terms of its layers, parameterization, and neuron connectivity. To unify these architectures under a common representation, we convert each of them into a graph structure. These resulting *model-graphs* abstract away architectural differences. Once in graph form, we employ a GNN to process and learn over these model-graphs. The GNN serves as a harmonizing function that operates over all trainable parameters across heterogeneous architectures within a shared graph-based representation space. This allows the models to co-train, exchange knowledge, and generalize more effectively—particularly under domain shifts or non-IID training conditions.

Our **uGNN** framework consists of three core steps: (i) converting heterogeneous architectures into graph representations, (ii) unifying these graph representations into a common graph-based learning space, and (iii) defining a GNN-driven optimization and training procedure. Steps (ii) and (iii) are illustrated in **Fig. 1-a**, while step (i) is shown in **Fig. 1-b**. Each step is detailed below.

Transforming Heterogeneous Architectures to Graphs. Here, we represent a DNN as a graph with nodes corresponding to DNN neurons and edges corresponding to the connections between DNN neurons [21]. While node features store the current activation and the bias of the corresponding neuron, edge features store the weight between two neurons. Additionally, we store activation functions as node features, as seen in **Fig. 1-b**, typically chosen from a pre-defined dictionary of common nonlinearities. In the following section, we first explain the graph transformation of Euclidean models by explaining how we convert MLPs and CNNs into model-graphs. Next, we show how to extend this transformation to non-Euclidean models, i.e., GNNs.

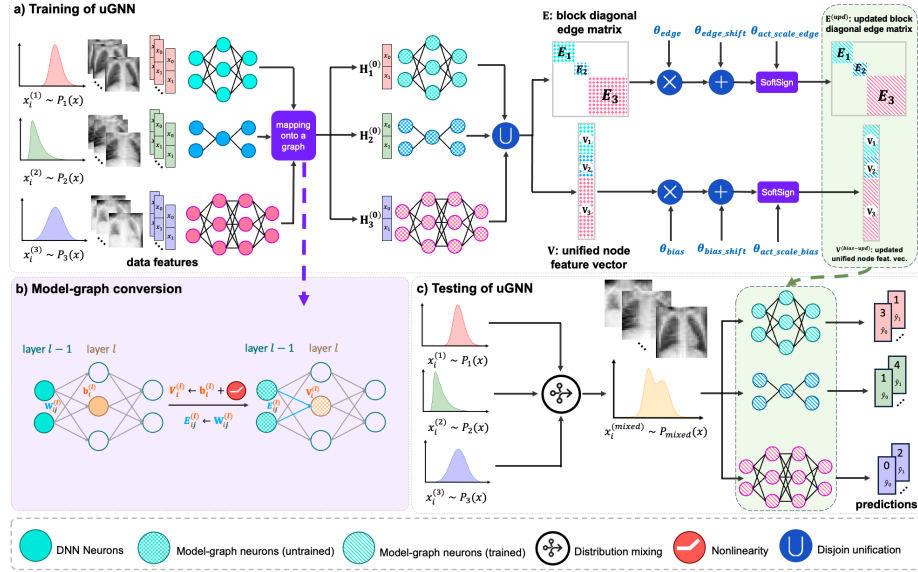


Fig. 1: Overview of the proposed uGNN framework. **(a) Training of uGNN:** Input data from multiple distributions $P_1(x)$, $P_2(x)$, $P_3(x)$ are fed into heterogeneous DNNs (MLPs and CNNs). Each DNN is mapped onto a model-graph with node features V_i (biases) and edge features E_i (weights). A unified graph is constructed through disjoint unification, where shared parameters (θ_{edge} , θ_{bias}) and corresponding shifts rescale and transform the edge and node features. **(b) Model-graph conversion:** DNN neurons are converted into model-graph neurons with edge features $E_{ij}^{(l)} \leftarrow W_{ij}^{(l)}$ and node features $V_i^{(l)} \leftarrow b_i^{(l)}$. Nonlinearities are stored in node features. **(c) Testing of uGNN:** During testing, the models are evaluated on a mixed distribution $P_{mixed}(x)$ to demonstrate improved generalization.

MLPs as Graphs. Consider an MLP with L layers, where each layer $l \in \{1, \dots, L\}$ contains d_l neurons. The weights and biases of the MLP are

represented as $\{\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \dots, \mathbf{W}^{(L)}\}$ and $\{\mathbf{b}^{(1)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(L)}\}$, respectively, where $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l \times 1}$. The total number of neurons across all layers (including input and output layers) is $n = \sum_{l=0}^L d_l$, with d_0 represents the number of input neurons. We construct a model-graph $G = (\mathbf{V}, \mathbf{E})$, where $\mathbf{V} \in \mathbb{R}^{n \times d_v}$ node features, and $\mathbf{E} \in \mathbb{R}^{n \times n \times d_e}$ represents edge features. Each node corresponds to a single neuron, and edges represent the weighted connections between neurons in consecutive layers. Importantly, due to the feedforward structure of MLPs, these connections form a *directed acyclic graph (DAG)*, ensuring no circular dependencies exist between nodes.

MLP Node Features. As depicted in **Fig. 1-b**, each node corresponds to a neuron in some layer l . The feature vector for a node $v_j^{(l)}$, representing the j -th neuron in layer l , is initialized from its corresponding bias term $\mathbf{V}_i = \mathbf{b}_j^{(l)}$ where $i = \text{Index}(l, j)$ maps the layer index l and neuron index j within that layer to a unique global node index i in $\mathbf{V}$.

MLP Edge Features. For edges, consider the connection from neuron $v_m^{(l-1)}$ in layer $(l-1)$ to neuron $v_j^{(l)}$ in layer l as visualized in **Fig. 1-b**. The corresponding weight $\mathbf{W}_{jm}^{(l)}$ serves as the edge feature. Let $i = \text{Index}(l, j)$ be the global node index of neuron j in layer l , and $k = \text{Index}(l-1, m)$ the global node index of neuron m in layer $(l-1)$. Then: $\mathbf{E}_{ik} = \mathbf{W}_{jm}^{(l)}$.

CNNs as Graphs. We now extend our graph representation from MLPs to CNNs. Consider a CNN architecture consisting of convolutional and fully connected (linear) layers. Each convolutional layer transforms an input feature map of size $C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}$ into an output feature map $C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$, where $C_{\text{in}}/C_{\text{out}}$ are the input/output channels and $H_{\text{in}}, W_{\text{in}}, H_{\text{out}}, W_{\text{out}}$ are the input/output spatial dimensions. L denotes the total number of layers (counting both convolutional and fully connected layers), and n the total number of “neuronal units” (i.e., spatial positions across all feature maps at all layers, plus the eventual neurons in fully connected layers).

We construct a model-graph $G = (\mathbf{V}, \mathbf{E})$ for the CNN in a manner analogous to the MLP case. Nodes now correspond to individual spatial locations in each layer’s feature map, and edges represent the connections defined by receptive fields across layers. Formally, $\mathbf{V} \in \mathbb{R}^{n \times d_v}$ denotes the node feature matrix, and $\mathbf{E} \in \mathbb{R}^{n \times n \times d_e}$ denotes the edge feature tensor.

CNN Node Features. Each node represents a single neuron at a particular spatial position and channel in a given layer. For an input layer (e.g., the raw image), each pixel position and channel is treated as a node. For a convolutional layer l , each output neuron corresponds to a spatial location (i, j) and channel c_{out} , yielding a node $v_{c_{\text{out}}, i, j}^{(l)}$. The bias parameters of the convolutional filter are used to initialize node features. Specifically, if $\mathbf{b}^{(l)} \in \mathbb{R}^{C_{\text{out}} \times 1}$ is the bias vector for layer l , each node corresponding to output channel c_{out} inherits its node feature from $\mathbf{b}_{c_{\text{out}}}^{(l)}$. Thus, $\mathbf{V}_u = \mathbf{b}_{c_{\text{out}}}^{(l)}$, $u = \text{Index}(l, c_{\text{out}}, i, j)$ where $\text{Index}(\cdot)$ provides a unique global node index for each spatial location and channel combination. If the network includes fully connected (linear) layers at the end, each neuron in these layers is also treated as a node, whose feature vector is initialized from the

corresponding bias term in the linear layer’s bias vector, analogous to the MLP case.

CNN Edge Features. Edges capture how each neuron in layer l is computed from a local receptive field of neurons in layer $(l - 1)$. For a convolutional layer, the kernel weights $\mathbf{W}^{(l)} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w}$ define connections from a $K_h \times K_w$ patch of neurons in the previous layer’s feature maps to a single output neuron. Thus, for each node $v_{c_{\text{out}}, i, j}^{(l)}$, we link it to nodes representing the patch (i', j') and channel c_{in} in layer $(l - 1)$ that lie within its receptive field. The corresponding edge feature is the kernel weight $\mathbf{W}_{c_{\text{out}}, c_{\text{in}}, \Delta i, \Delta j}^{(l)}$, where $(\Delta i, \Delta j)$ indexes the kernel element mapping from position $(i - \Delta i, j - \Delta j)$ in the previous layer to (i, j) in the current layer:

$$\mathbf{E}_{uv} = \mathbf{W}_{c_{\text{out}}, c_{\text{in}}, \Delta i, \Delta j}^{(l)}, \quad u = \text{Index}(l, c_{\text{out}}, i, j), \quad v = \text{Index}(l-1, c_{\text{in}}, i-\Delta i, j-\Delta j)$$

For subsequent fully connected layers, edges are constructed similarly to the MLP case: each output neuron in the fully connected layer connects to all neurons in the previous layer, and edge features store the corresponding linear weights.

Graph Connectivity and DAG Structure. As for MLPs, the CNN’s feedforward nature ensures that the resulting graph is a DAG. Nodes in earlier layers connect forward to nodes in later layers, but no edges exist backward or laterally that would introduce cycles:

$$\mathbf{A}_{uv} = \begin{cases} 1 & \text{if } v_u \text{ is a node in layer } l, v_v \text{ is a node in layer } (l - 1), \\ & \text{and } v_v \text{ lies within the receptive field of } v_u, \\ 0 & \text{otherwise.} \end{cases}$$

GNNs as Graphs. We now extend our graph representation to GNNs themselves. Unlike MLPs and CNNs, GNNs already operate on graph-structured data, but we can still represent their architectures as meta-graphs for unified learning. In this formulation, each node in the model-graph represents a node in the input graph at a specific layer of the GNN, while edges represent the aggregation of messages across layers.

Consider a GNN with L layers operating on input graphs with N nodes. For each layer l , let $\mathbf{H}^{(l)} \in \mathbb{R}^{N \times d_l}$ denote the node embeddings at that layer, where d_l is the embedding dimension. The GNN’s message passing can be abstracted as a sequence of transformations: $\mathbf{H}^{(l)} = \sigma(\hat{\mathbf{A}}\mathbf{H}^{(l-1)}\mathbf{W}^{(l)})$ where $\hat{\mathbf{A}}$ is the normalized adjacency matrix (potentially with self-loops), $\mathbf{W}^{(l)}$ is the trainable weight matrix at layer l , and σ is a nonlinearity such as ReLU.

GNN Node Features. In our model-graph, each node represents a tuple (v, l) where v is a node in the input graph and l is a layer index. The node feature combines the bias associated with that layer and the embedding dimension: $\mathbf{V}_{(v, l)} = \mathbf{b}^{(l)}$ where $\mathbf{b}^{(l)}$ is the bias vector of layer l (broadcasted across all input nodes).

GNN Edge Features. Edges in the model-graph represent message passing between nodes across layers. For nodes u and v in the input graph connected by an edge, we add edges in the model-graph between $(u, l - 1)$ and (v, l) for

each layer l . We define the edge features correspond to the weight matrix $\mathbf{W}^{(l)}$ applied during aggregation as $\mathbf{E}_{(v,l),(u,l-1)} = \mathbf{W}^{(l)}$

Unifying Model-Graphs into a Common Learning Space. Consider a collection of m heterogeneous models $\{M_1, M_2, \dots, M_m\}$, each represented by its own model-graph $(\mathbf{V}_i, \mathbf{E}_i)$ as described above. We construct a unified representation by forming a disjoint union of all these graphs (**Fig. 1-a**): $\mathbf{V} = \bigcup_{i=1}^m \mathbf{V}_i$, $\mathbf{E} = \bigcup_{i=1}^m \mathbf{E}_i$. This operation creates a larger graph $G = (\mathbf{V}, \mathbf{E})$ containing all nodes and edges from each individual model-graph, without introducing edges between different models. Consequently, G can be viewed as a block-diagonal adjacency structure, where each block corresponds to the topology of a single model. By training a GNN on this unified graph G , we process all subgraphs (each corresponding to a different model) simultaneously with a single set of shared parameters. This enables the GNN to learn a common representation space, potentially capturing model-invariant properties and facilitating knowledge transfer among the diverse set of model architectures.

Parametrization. Each model’s edges correspond to parameterized connections (weights) and each node corresponds to a neuron with an associated bias. To replicate these computations, we introduce shared learnable parameters $(\theta_{\text{edge}}, \theta_{\text{bias}}, \theta_{\text{edge_shift}}, \theta_{\text{bias_shift}}, \theta_{\text{act_scale_edge}}, \theta_{\text{act_scale_bias}})$ that adjust and transform the original edges and biases from all models simultaneously. The pipeline of these parameters are visually depicted in **Fig. 1-a**. Specifically, let $\theta_{\text{edge}} \in \mathbb{R}^{k_{\text{edge}}}$ and $\theta_{\text{edge_shift}} \in \mathbb{R}^{k_{\text{edge}}}$ be learnable parameters used to rescale and shift the edge features (corresponding to underlying model weights). We randomly partition edges into k_{edge} groups, and each edge group shares a single scalar scale and shift parameter. For an edge (u, v) belonging to group $g(u, v)$, we define the updated edge features as $\mathbf{E}_{u,v}^{(\text{upd})} = \text{SoftSign}(\mathbf{E}_{u,v} \cdot \theta_{\text{edge}}^{(g(u,v))} + \theta_{\text{edge_shift}}^{(g(u,v))}, \theta_{\text{act_scale_edge}})$ where $\text{SoftSign}(x, s)$ is an element-wise nonlinearity scaled by s defined as $\text{SoftSign}(x, s) = \frac{s \cdot x}{s + |x|}$. Note that $\mathbf{E}_{u,v}$ denotes the original edge feature (e.g., a weight) and $\theta_{\text{act_scale_edge}}$ is a scalar controlling the softsign scaling. Similarly, node biases are grouped into k_{bias} groups. Each node bias group has its own scale and shift parameters $(\theta_{\text{bias}}, \theta_{\text{bias_shift}}) \in \mathbb{R}^{k_{\text{bias}}}$. For a node v in group $h(v)$, we update its bias as $\mathbf{V}_v^{(\text{bias-upd})} = \text{SoftSign}(\mathbf{V}_v \cdot \theta_{\text{bias}}^{(h(v))} + \theta_{\text{bias_shift}}^{(h(v))}, \theta_{\text{act_scale_bias}})$ where $\mathbf{V}_v$ is the original bias feature of the node v (derived from the underlying model parameters), and $\theta_{\text{act_scale_bias}}$ adjusts the scaling of the SoftSign function. By applying these transformations, the GNN aligns all underlying model parameters (edges and biases) into a common parametrization that can be adjusted during training.

Message Passing and Activation. After updating edges and node biases as described, the forward pass emulates the feedforward computation of the underlying models layer-by-layer. Let $\mathbf{H}^{(\ell)} \in \mathbb{R}^{n \times 1}$ represent the node activations in layer ℓ , where layers are implicitly defined by the topological ordering in G . As illustrated in **Fig. 1-a**, for the first layer, we set $\mathbf{H}^{(0)} = \mathbf{X}$ where $\mathbf{X}$ are input features for input nodes (and zeros for other nodes until their layer’s turn). At each subsequent layer $\ell > 0$, we use the updated edge features to aggregate

incoming signals $\mathbf{Z}_v^{(\ell)} = \sum_{v:\mathbf{A}_u v=1} \mathbf{E}_{uv}^{(\text{upd})} \cdot \mathbf{H}_u^{(\ell-1)}$ where $\mathbf{E}_{uv}^{(\text{upd})}$ are the updated edge features from node u to v , and $\mathbf{H}_u^{(\ell-1)}$ is the activation of node u from the previous layer. This operation is analogous to performing a matrix multiplication of weights with activations in an MLP or applying convolution-like operations in a CNN. We then add the updated node bias $\mathbf{Z}_v^{(\ell)} \leftarrow \mathbf{Z}_v^{(\ell)} + \mathbf{V}_v^{(\text{bias-upd})}$ where $\mathbf{V}_v^{(\text{bias-upd})}$ is the updated bias for node i .

Layer-by-Layer Computation and Model Emulation. By iterating this process from $\ell = 1$ up to the deepest layer present in any model subgraph, we replicate the forward pass of each underlying model within this unified GNN framework. The parameterized transformations $(\theta_{\text{edge}}, \theta_{\text{bias}})$ and associated shifts ensure that our uGNN can adaptively rescale and align parameters so that the resulting node outputs match the computations of the target models.

Loss Computation and Training. Following the uGNN forward pass and the computation of the node activations at the final layer, $\mathbf{H}^{(L_i)}$, we can extract the predictions of each original model M_i . Since all models share the same classification task, final-layer activations, $\mathbf{H}^{(L_i)}$, corresponds to the predicted logits by model M_i . Let $\mathbf{Y}^{(i)}$ be the ground-truth labels associated with model M_i 's data. For classification, we use a standard loss function such as cross-entropy: $\mathcal{L}_i = -\sum_c Y_c^{(i)} \log \hat{Y}_c^{(i)}$, where $\hat{\mathbf{Y}}^{(i)}$ are the softmax-normalized predictions derived from $\mathbf{H}^{(L_i)}$. Each $\mathcal{L}_i$ measures how well the uGNN's predictions match the ground-truth labels for the i -th model's subgraph. To train the GNN so that it can accurately emulate all m original models' computations, we combine the individual losses into a single scalar objective by computing a weighted sum $\mathcal{L} = \sum_{i=1}^m \alpha_i \mathcal{L}_i$, where $\sum_{i=1}^m \alpha_i = 1$ and $\alpha_i \geq 0$. Here, α_i represents the weight assigned to the i -th model's loss, reflecting its relative importance during training. These weights can be learned or set manually based on specific criteria, such as dataset size or task complexity. We minimize $\mathcal{L}$ with respect to all GNN parameters $(\theta_{\text{edge}}, \theta_{\text{bias}}, \theta_{\text{edge_shift}}, \theta_{\text{bias_shift}}, \theta_{\text{act_scale_edge}}, \theta_{\text{act_scale_bias}})$ using standard gradient-based optimization. By doing so, our uGNN learns a unified parametrization that can accurately reconstruct the forward pass of each original model, leading to a single learned representation that successfully performs the classification task for all model-graphs.

4 Experiments & Results

To evaluate the generalization capabilities of our unified learning paradigm, we design our experimental setup as follows. During training, each model is associated with a particular distribution: $x_i^{(k)} \sim P_k(x)$, where $P_k(x)$ denotes the k -th distributional cluster derived from the data. At testing time, as illustrated in **Fig. 1-c**, each model encounters test samples drawn from a mixture of distributions: $x_i^{(\text{mixed})} \sim P_{\text{mixed}}(x)$, where $P_{\text{mixed}}(x) = \sum_{k=1}^K P_k(x)$, and K is the number of clusters. This setup directly tests the ability of models to generalize beyond their individually seen distributions.

We consider two training scenarios: **(1) Individual Training:** Each model is trained independently on a single cluster, without knowledge of the other clusters

(i.e., distributions). **(2) uGNN Training:** Using our unified GNN framework, we *jointly* train multiple model-graphs, each independently accessing data with a distinct distribution (**Fig. 1-a**). Although each model-graph individually accesses only one specific distribution, the shared parameters ($\theta_{\text{edge}}, \theta_{\text{bias}}$) within the uGNN facilitate indirect knowledge transfer across distributions. In this manner, the uGNN leverages signals from multiple distributions without explicitly sharing them between models, potentially enhancing generalization.

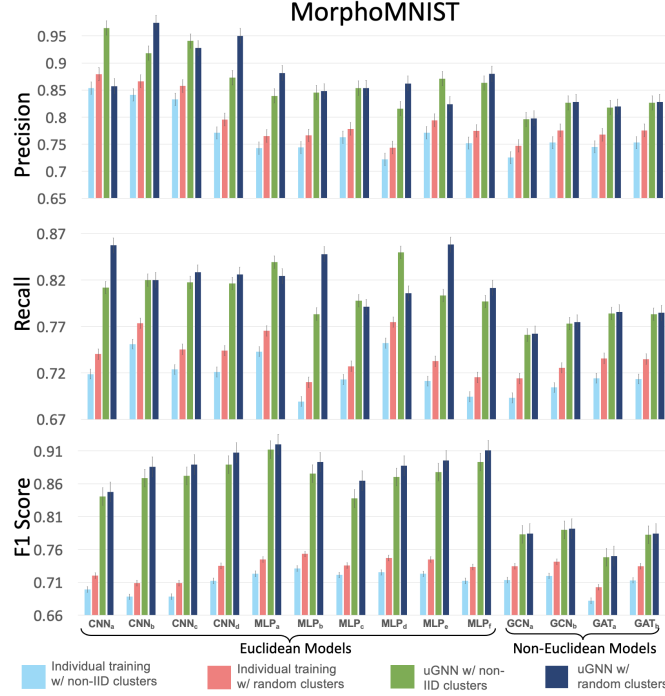


Fig. 2: Comparison of testing performance on the MorphoMNIST dataset between the baseline approach and our uGNN across six Euclidean and four non-Euclidean models. Each architecture has been trained on either plain, thinned or thickened perturbations.

Our experiments focus on the BreastMNIST and PneumoniaMNIST datasets from **MedMNIST** [22] and **MorphoMNIST** [23]. For each dataset, we have two clustering setup: **(1) Non-IID:** To generate non-IID clusters, we first reduce the dimensionality of the 28×28 images using Principal Component Analysis (PCA) with 50 components, followed by k -means clustering on the PCA-transformed features with a distinct *random seed* for each fold, ensuring diversity and reproducibility in the clustering process. The resulting clusters represent distinct distributions. **(2) Random:** In random clustering, we randomly create

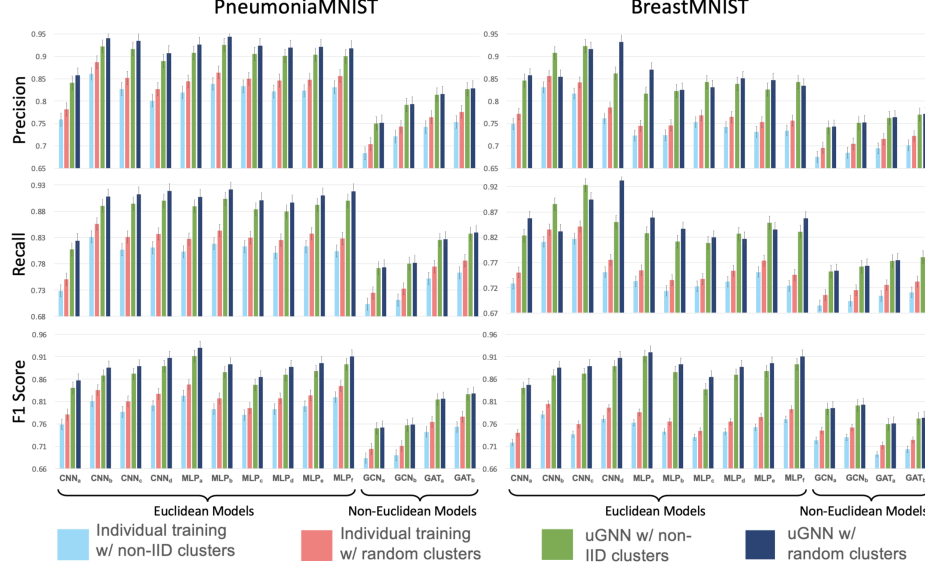


Fig. 3: Comparison of testing performance for six Euclidean and four non-Euclidean models on the PneumoniaMNIST and BreastMNIST datasets, evaluated using the baseline approach and our uGNN.

clusters from samples in the dataset so that clusters do not exhibit any engineered distribution shift between them.

During training, each cluster is assigned to a separate model-graph, allowing the uGNN framework to unify and refine parameters across these diverse data subsets.

In all our experiments, we set k_{edge} and k_{bias} to the total number of edges and nodes in the unified graph G , respectively. As a result, each edge and bias group is associated with a single parameter. To ensure a fair comparison, we use the same training setup for both individual models and the uGNN framework: the AdamW optimizer with identical learning rate and weight decay settings, the same number of training epochs, and the same learning rate scheduler. The initialization of underlying models is also identical for both approaches. To ensure stable training, we initialize both the underlying models and the uGNN parameters carefully. We initialize an underlying model identically for both uGNN and baseline training versions. For the uGNN parameters, aggressive schemes like Gaussian or Kaiming initialization [24] destabilize training by causing excessive updates to edge and node features. Therefore, we adopt the following conservative initialization: $\theta_{\text{edge}} = \mathbf{1}_{k_{\text{edge}}}$, $\theta_{\text{edge_shift}} = \mathbf{0}_{k_{\text{edge}}}$, $\theta_{\text{bias}} = \mathbf{1}_{k_{\text{bias}}}$, $\theta_{\text{bias_shift}} = \mathbf{0}_{k_{\text{bias}}}$, $\theta_{\text{edge_act_scale}} = 1$, $\theta_{\text{bias_act_scale}} = 1$. This initialization ensures minimal initial updates to edge and node features.

As shown in **Fig. 2**, we evaluate on **MorphoMNIST** [23], where DNNs are trained on plain, thinned, or thickened digits, and tested on a mixed distribu-

tion. uGNN consistently improves generalization across all models compared to individual training. **Fig. 3** compares individual training with uGNN on **PneumoniaMNIST** and **BreastMNIST**. uGNN outperforms individual training in all cases, demonstrating significant performance gains on real-world clinical datasets—highlighting its potential utility in medical imaging applications. Furthermore, we observe that models trained on random client clusters perform better than those trained on non-IID clusters. This is because random clustering does not introduce additional domain shifts beyond the dataset’s inherent distribution, whereas non-IID clustering exacerbates domain discrepancies by design.

5 Discussion & Conclusion

In this work, we introduced a unified learning framework that brings together heterogeneous neural network architectures—namely MLPs, CNNs, and GNNs—within a shared graph-based representation for joint training under a common parameter space. By converting diverse architectures into graph structures, our approach enables indirect knowledge transfer across models, even when they operate in fundamentally different (Euclidean vs. non-Euclidean) spaces. This leads to improved generalization, particularly under domain shifts and distributional heterogeneity. Our framework proves effective in both homogeneous and heterogeneous model settings by harmonizing architectural differences and enabling collaborative learning. Looking ahead, we aim to extend this framework to support *multi-modality* and *multi-task learning*, paving the way for unified, distribution- and architecture-agnostic learning paradigms. Such capabilities are especially promising for collaborative settings like federated learning, with high-impact applications in domains such as precision medicine.

References

1. Litjens, G., Kooi, T., Bejnordi, B.E., Setio, A.A.A., Ciompi, F., Ghafoorian, M., Van Der Laak, J.A., Van Ginneken, B., Sánchez, C.I.: A survey on deep learning in medical image analysis. *Medical image analysis* **42** (2017) 60–88
2. Shen, D., Wu, G., Suk, H.I.: Deep learning in medical image analysis. *Annual review of biomedical engineering* **19** (2017) 221–248
3. Yan, K., Wang, X., Lu, L., Summers, R.M.: Deeplesion: automated mining of large-scale lesion annotations and universal lesion detection with deep learning. *Journal of medical imaging* **5** (2018) 036501–036501
4. Yoon, J.S., Oh, K., Shin, Y., Mazurowski, M.A., Suk, H.I.: Domain generalization for medical image analysis: A review. *Proceedings of the IEEE* (2024)
5. Nguyen, T., Do, K., Duong, B., Nguyen, T.: Domain generalisation via risk distribution matching. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. (2024) 2790–2799
6. Li, L., Fan, Y., Tse, M., Lin, K.Y.: A review of applications in federated learning. *Computers & Industrial Engineering* **149** (2020) 106854

7. Kairouz, P., McMahan, H.B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A.N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R., et al.: Advances and open problems in federated learning. *Foundations and trends® in machine learning* **14** (2021) 1–210
8. Wen, Z., Zhang, X., Ye, C.: Source-free domain adaptation for medical image segmentation via selectively updated mean teacher. In: *International Conference on Information Processing in Medical Imaging*, Springer (2023) 225–236
9. Kamnitsas, K., Baumgartner, C., Ledig, C., Newcombe, V., Simpson, J., Kane, A., Menon, D., Nori, A., Criminisi, A., Rueckert, D., et al.: Unsupervised domain adaptation in brain lesion segmentation with adversarial networks. In: *Information Processing in Medical Imaging: 25th International Conference, IPMI 2017, Boone, NC, USA, June 25–30, 2017, Proceedings 25*, Springer (2017) 597–609
10. Hinton, G.: Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015)
11. Yang, Q., Liu, Y., Chen, T., Tong, Y.: Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)* **10** (2019) 1–19
12. Sohan, M.F., Basalamah, A.: A systematic review on federated learning in medical image analysis. *IEEE Access* **11** (2023) 28628–28644
13. Rehman, M.H.u., Hugo Lopez Pinaya, W., Nachev, P., Teo, J.T., Ourselin, S., Cardoso, M.J.: Federated learning for medical imaging radiology. *The British Journal of Radiology* **96** (2023) 20220890
14. Ganin, Y., Lempitsky, V.: Unsupervised domain adaptation by backpropagation. In: *International conference on machine learning*, PMLR (2015) 1180–1189
15. Zhou, K., Yang, Y., Hospedales, T., Xiang, T.: Learning to generate novel domains for domain generalization. In: *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XVI 16*, Springer (2020) 561–578
16. Singh, S.P., Jaggi, M.: Model fusion via optimal transport. *Advances in Neural Information Processing Systems* **33** (2020) 22045–22055
17. Bronstein, M.M., Bruna, J., LeCun, Y., Szlam, A., Vandergheynst, P.: Geometric deep learning: going beyond euclidean data. *IEEE Signal Processing Magazine* **34** (2017) 18–42
18. Wang, J.: Review of graph neural networks for medical image. *EAI Endorsed Transactions on e-Learning* **9** (2023)
19. Ravinder, M., Saluja, G., Allabun, S., Alqahtani, M.S., Abbas, M., Othman, M., Soufiene, B.O.: Enhanced brain tumor classification using graph convolutional neural network architecture. *Scientific Reports* **13** (2023) 14938
20. Zou, Y., Pesaranhader, A., Song, Z., Verma, A., Buckeridge, D.L., Li, Y.: Modeling electronic health record data using an end-to-end knowledge-graph-informed topic model. *Scientific Reports* **12** (2022) 17868
21. Kofinas, M., Knyazev, B., Zhang, Y., Chen, Y., Burghouts, G.J., Gavves, E., Snoek, C.G.M., Zhang, D.W.: Graph neural networks for learning equivariant representations of neural networks. In: *The Twelfth International Conference on Learning Representations*. (2024)
22. Yang, J., Shi, R., Wei, D., Liu, Z., Zhao, L., Ke, B., Pfister, H., Ni, B.: Medmnist v2—a large-scale lightweight benchmark for 2d and 3d biomedical image classification. *Scientific Data* **10** (2023) 41
23. Castro, D.C., Tan, J., Kainz, B., Konukoglu, E., Glocker, B.: Morpho-MNIST: Quantitative assessment and diagnostics for representation learning. *Journal of Machine Learning Research* **20** (2019)

24. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In: Proceedings of the IEEE international conference on computer vision. (2015) 1026–1034